

TECHNICAL BREAKDOWN OF THE PROJECT

IoT Based Color Cap Monitor

Component Information and Function

The system uses a **modular architecture** split into a processing layer and a sensory layer to achieve stable operations:

ESP 32 Development Board: Acts as the central microcontroller hub. It handles the main code execution, processes the high-speed frequency data from the color sensor, controls the status indicator, and uses its internal 2.4 GHz radio to push live data to the cloud via Wi-Fi.



ESP32 Development Board

TCS 3200 Color Sensor: The primary optical sensor that scans the target surface of the bottle caps to detect color variations.



irasaelec.com

TCS3200 Color Sensor

H C05 Bluetooth Module: Handles the short-range wireless communication to the mobile application. It operates independently via Hardware Serial (UART2 on GPIO 16/17).



HC-05 Bluetooth Module

KEYES KY-009 RGB LED Module: A single surface-mount (SMD 5050) component that replaces multiple individual LEDs. It dynamically blends colors (Red, Yellow, Blue, White) to instantly display the current system or calibration state.



KEYES KY-009 RGB LED Module

Technical Analysis of the Color Detection System

The core of the system relies on optical-to-digital signal conversion managed by the **TCS3200 color sensor** and processed via the **ESP32 microcontroller**. Instead of outputting standard analog voltages, the sensor converts light intensity into an electrical square wave whose frequency corresponds directly to the brightness of the light hitting it.

1. Internal Sensor Architecture & Photodiode Array

The TCS3200 integrated circuit houses a matrix of 8×8 photodiodes (64 photodiodes total) grouped into four distinct functional sets:

- 16 Photodiodes with Red Filters: sensitive to longer wavelengths of light (~600–700 nm).
- 16 Photodiodes with Green Filters: sensitive to mid-spectrum wavelengths (~500–600 nm).
- 16 Photodiodes with Blue Filters: sensitive to shorter wavelengths (~400–500 nm).
- 16 Photodiodes with Clear Filters: no optical filtering; measures total ambient/reflected white light intensity.

The sensor uses two control lines, S2 and S3 (Select Pins), which act as a digital multiplexer. By switching the states of these pins (LOW/LOW for Red, HIGH/HIGH for Green, LOW/HIGH for Blue), the ESP32 sequentially selects which photodiode array's internal output is routed to the processing pin.

2. Light Intensity-to-Frequency Conversion

- 1 **Optical Reflection:** The onboard white LEDs illuminate the target surface (the moving bottle cap). The surface absorbs certain wavelengths and reflects others based on its physical color profile.
- 2 **Current Generation:** The reflected light passes through the selected photodiode filters. The silicon junctions generate a micro-ampere analog current that scales linearly with the intensity of the light passing through that color filter.
- 3 **Square-Wave Output:** The internal current-to-frequency converter transforms this analog current into a clean 50% duty cycle square wave. The frequency (f_{out}) of this square wave is directly proportional to the light intensity.
- 4 **Frequency Scaling (S0/S1 Pins):** Pins S0 and S1 configure the internal output scaling to match the microcontroller's sampling speed. For the ESP32, scaling is configured to 20% (S0 = HIGH, S1 = LOW), shifting the typical output range up to ~100–120 kHz, which is easily read by the ESP32's hardware timers without blocking main thread execution loops.

3. ESP32 Frequency Capture & Pulse Counting

The ESP32 reads the incoming pulse train on its designated input GPIO pin. The software samples each channel sequentially:

- 1 The pins S2 and S3 are set to read the Red channel. The system waits a brief stabilization window, then counts the number of rising edges within a fixed duration (e.g., 20 to 50 milliseconds) using an interrupt routine or pulseIn() tracking.
- 2 The select pins immediately cycle to the Green channel, and the pulse accumulation is repeated.
- 3 The select pins cycle to the Blue channel, capturing the final color component.

This loop generates three raw numeric values: R_{freq} , G_{freq} , and B_{freq} . On this hardware configuration, a lower raw frequency time-span value actually represents higher color intensity, because the sensor responds faster to brighter light components.

4. Mathematical Vector Distance Classification Logic & Embedded Optimization

To achieve real-time classification without brittle, hardcoded conditional blocks, the firmware implements a localized **Nearest Neighbor Classification Algorithm** operating in a three-dimensional optical coordinate space.

Coordinate Space Mapping

When the system is calibrated in "Teach Mode", it establishes reference coordinate vectors for up to four target classes:

$$C_i = [R_i , G_i , B_i]$$

where $i \in \{0, 1, 2, 3\}$, and the components represent the baseline time durations (in microseconds) captured via the sensor's current-to-frequency converter.

When a bottle cap passes the scanning window in continuous detection mode, the ESP32 captures a live target vector:

$$T = [R_t , G_t , B_t]$$

Note: Because the low-level driver utilizes the blocking pulseIn(OUT, LOW) function, the vector components represent the period (T) of the generated square wave. Consequently, there is an inverse relationship to light intensity — smaller values indicate a higher frequency output, corresponding to a brighter, more reflective surface profile.

Optimized Squared Euclidean Distance Formula

In standard coordinate geometry, the spatial distance (d) between the live target and a reference profile is calculated using the standard Euclidean metric:

$$d = \sqrt{[(R_t - R_i)^2 + (G_t - G_i)^2 + (B_t - B_i)^2]}$$

However, executing floating-point square root operations (sqrt()) inside a rapid execution loop introduces severe computational overhead, consuming valuable CPU clock cycles on the microcontroller. Because the square root function is strictly monotonic for all non-negative real numbers, the inequality relationship is preserved without it:

$$\text{If } d_a^2 < d_b^2 \text{ then } d_a < d_b$$

To exploit this, the firmware uses an optimized **Squared Euclidean Distance (d²)** calculation, executing strictly with fast integer arithmetic:

$$d^2 = (R_t - R_i)^2 + (G_t - G_i)^2 + (B_t - B_i)^2$$

In the firmware implementation, this is translated into delta-accumulation registers:

```
long dr = r - colors[i].r;
long dg = g - colors[i].g;
long db = b - colors[i].b;
long distance = dr * dr + dg * dg + db * db;
```

Decision Boundary & Threshold Verification

The internal matching engine iterates through the calibrated database to find the absolute minimum spatial variance:

$$\text{bestDistance} = \min(d_0^2 , d_1^2 , d_2^2 , d_3^2)$$

Once the closest vector index is identified, the system applies a strict hyperspherical decision boundary. The calculated bestDistance is evaluated against a predefined scalar threshold (20,000 μs²):

$$\begin{aligned} \text{Classification} = & \\ & \text{Index } i, \quad \text{if } \text{bestDistance} \leq \text{THRESHOLD} \\ & -1 \text{ (Unknown / Defect),} \quad \text{if } \text{bestDistance} > \text{THRESHOLD} \end{aligned}$$

If the minimum distance falls within this hypersphere, the system validates the color match, triggers the corresponding multi-pin digital mixing on the KY-009 RGB module, and pushes the telemetry log packet to the HiveMQ cloud broker. If it exceeds the boundary, it flags an outlier anomaly, isolating defective or uncalibrated caps instantly.

Wireless Operations & Cloud Platform

To eliminate interference and system lag, a **Decoupled Radio Architecture** is implemented:

- **The Problem:** The ESP32's internal antenna shares both Wi-Fi and Bluetooth. Running both concurrently causes heavy resource conflicts, latency, and disconnections.
- **The Architecture:** The internal Bluetooth is kept completely disabled ("dead") so 100% of the internal antenna is dedicated to Wi-Fi. The Bluetooth protocol stack is entirely offloaded to the external HC-05 hardware.

Local Mode (Mobile App)

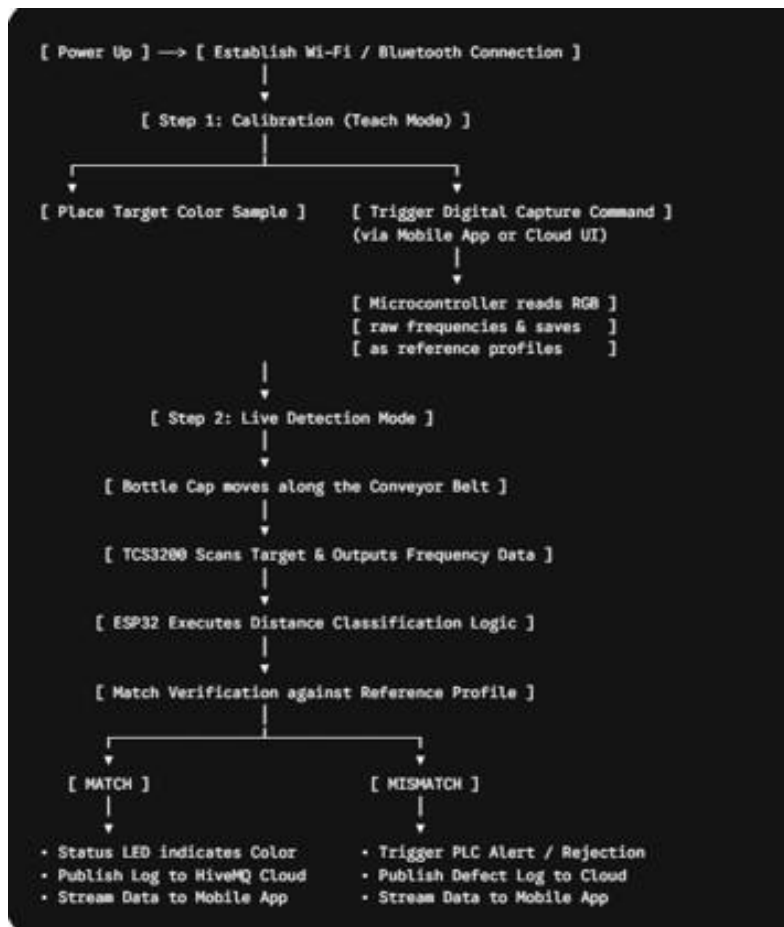
How it works: The smartphone app was built using MIT App Inventor. It communicates via Serial Port Protocol (SPP) with the HC-05 module. It allows the operator to execute calibration controls and view live diagnostics right next to the machine. There are buttons like "c1", "c2", "c3", "c4" to calibrate each color, "D" to start real-time detection, and "stop" to stop the detection. The app also shows which color got detected and which color was calibrated.

Remote Cloud Mode (Website Interface)

Platform Name: Color Cap Monitor web interface (connected through HiveMQ Cloud Broker).

How it works: The hardware and software layers communicate seamlessly over a secure pipeline. The ESP32 utilizes the MQTT IoT Telemetry Protocol to establish a persistent wireless link. Every time a bottle cap color is scanned and classified, the microcontroller packages the raw metrics and results, publishing them as live telemetry logs to the HiveMQ Cloud Broker. The Color Cap Monitor website continuously subscribes to this data feed, instantly refreshing the remote user dashboard so supervisors can track production metrics and defect flags in real time from any location.

System Operation Flow



Overall system operation flow, from power-up through calibration and live detection.